Fonic HiFi SaaS Implementation Plan

This document outlines the detailed implementation plan for adding a privacy-first SaaS subscription model to the Fonic HiFi iOS application. The approach maintains the app's core privacy principles while creating a sustainable revenue model.

SaaS Subscription Infrastructure

Step 1: Subscription Model Foundation

Detailed technical explanation of what we're accomplishing in this step

We'll establish the core subscription model architecture, including subscription tiers, feature flags, and the foundation for StoreKit 2 integration. This will provide the framework for implementing privacy-preserving subscriptions without requiring user accounts or data collection.

Task Breakdown

Define Subscription Tiers

- Create enum for subscription levels (Free, Premium)
- /FonicHiFi/Domain/Models/SubscriptionTier.swift
- Create

```
enum SubscriptionTier: String, Codable, CaseIterable {
  case free
  case premium

  var displayName: String {
    switch self {
    case .free: return "Core"
    case .premium: return "Premium"
    }
  }

  var features: [FeatureFlag] {
    switch self {
    case .free:
      return [.basicPlayback, .standardFormats, .basicEqualizer, .localLibrary]
    case .premium:
      return FeatureFlag.allCases
    }
```

```
    }
}
```

Implement Feature Flag System

- Create enum for feature flags with tier requirements
- /FonicHiFi/Domain/Models/FeatureFlag.swift
- Create

```
enum FeatureFlag: String, Codable, CaseIterable {
    // Free features
    case basicPlayback
    case standardFormats
    case basicEqualizer
    case localLibrary

    // Premium features
    case advancedDSP
    case cloudIntegration
    case advancedMetadata
    case highResFormats
    case advancedWaveforms

    var requiredTier: SubscriptionTier {
        switch self {
        case .basicPlayback, .standardFormats, .basicEqualizer, .localLibrary:
            return .free
        default:
            return .premium
        }
    }

    var displayName: String {
        switch self {
        case .basicPlayback: return "Basic Playback"
        case .standardFormats: return "Standard Formats"
        case .basicEqualizer: return "10-Band Equalizer"
        case .localLibrary: return "Local Library"
        case .advancedDSP: return "Advanced Audio Processing"
        case .cloudIntegration: return "Cloud Storage Integration"
        case .advancedMetadata: return "Advanced Metadata Editing"
        case .highResFormats: return "High-Resolution Format Support"
        case .advancedWaveforms: return "Advanced Waveform Visualization"
        }
    }

    var description: String {
        switch self {
        case .basicPlayback: return "Play, pause, skip, and basic playback controls"
        case .standardFormats: return "Support for FLAC, ALAC, MP3, and AAC
```

```
formats"
        case .basicEqualizer: return "10-band equalizer with basic presets"
        case .localLibrary: return "Import and organize local music files"
        case .advancedDSP: return "Room correction, spatial audio, and advanced
processing"
        case .cloudIntegration: return "Connect to Google Drive, Dropbox, and other
cloud storage"
        case .advancedMetadata: return "Batch editing and advanced metadata
management"
        case .highResFormats: return "DSD and high-resolution format support"
        case .advancedWaveforms: return "Enhanced waveform visualization and
analysis"
        }
    }
}
```

Create Subscription Product Models

- Define models for StoreKit 2 products
- /FonicHiFi/Domain/Models/SubscriptionProduct.swift
- Create

```
struct SubscriptionProduct: Identifiable, Equatable {
    let id: String
    let displayName: String
    let description: String
    let price: Decimal
    let tier: SubscriptionTier
    let duration: SubscriptionDuration

    // StoreKit 2 product reference
    var storeKitProduct: Product?
}

enum SubscriptionDuration: String, Codable {
    case monthly
    case yearly

    var displayName: String {
        switch self {
        case .monthly: return "Monthly"
        case .yearly: return "Yearly"
        }
    }
}
```

Set Up Subscription Configuration

- Create configuration for subscription products

- /FonicHiFi/Core/Subscription/SubscriptionConfiguration.swift
- Create

```
struct SubscriptionConfiguration {
  static let productIdentifiers = [
    "com.fonichifi.subscription.premium.monthly",
    "com.fonichifi.subscription.premium.yearly"
  ]

  static let defaultProducts: [SubscriptionProduct] = [
    SubscriptionProduct(
      id: "com.fonichifi.subscription.premium.monthly",
      displayName: "Premium Monthly",
      description: "Full access to all premium features",
      price: 4.99,
      tier: .premium,
      duration: .monthly
    ),
    SubscriptionProduct(
      id: "com.fonichifi.subscription.premium.yearly",
      displayName: "Premium Yearly",
      description: "Full access to all premium features",
      price: 49.99,
      tier: .premium,
      duration: .yearly
    )
  ]
}
```

Other Notes On Step 1: Subscription Model Foundation

- Ensure all subscription-related models are immutable and thread-safe
- Consider future expansion to additional tiers if needed

Step 2: StoreKit 2 Integration

Detailed technical explanation of what we're accomplishing in this step

We'll implement the StoreKit 2 integration for handling in-app purchases and subscriptions. This will provide the mechanism for users to subscribe to premium features while maintaining privacy by using Apple's anonymous subscription system.

Task Breakdown

Add StoreKit 2 Dependencies

- Configure project for StoreKit 2

- /FonicHiFi.xcodeproj/project.pbxproj
- Update
- /FonicHiFi/Resources/Info.plist
- Update

Create StoreKit Configuration

- Set up StoreKit configuration for testing
- /FonicHiFi/Resources/StoreKitConfiguration.storekit
- Create

Implement Store Service

- Create service for interacting with StoreKit 2
- /FonicHiFi/Core/Subscription/StoreService.swift
- Create

```
import StoreKit

actor StoreService: ObservableObject {
    @Published private(set) var products: [Product] = []
    @Published private(set) var purchasedProductIDs = Set<String>()

    private var productIDs = SubscriptionConfiguration.productIdentifiers
    private var updates: Task<Void, Error>? = nil

    init() {
        updates = observeTransactionUpdates()
    }

    deinit {
        updates?.cancel()
    }

    @MainActor
    func loadProducts() async throws {
        products = try await Product.products(for: productIDs)
    }

    func purchase(_ product: Product) async throws -> Transaction? {
        let result = try await product.purchase()

        switch result {
        case .success(let verification):
            let transaction = try checkVerified(verification)
            await updatePurchasedProducts()
            await transaction.finish()
            return transaction
        case .userCancelled:
```

```
            return nil
        case .pending:
            return nil
        @unknown default:
            return nil
        }
    }

    func checkVerified<T>(_ result: VerificationResult<T>) throws -> T {
        switch result {
        case .unverified:
            throw StoreError.failedVerification
        case .verified(let safe):
            return safe
        }
    }

    @MainActor
    func updatePurchasedProducts() async {
        for await result in Transaction.currentEntitlements {
            do {
                let transaction = try checkVerified(result)

                if transaction.revocationDate == nil {
                    purchasedProductIDs.insert(transaction.productID)
                } else {
                    purchasedProductIDs.remove(transaction.productID)
                }

            } catch {
                print("Transaction failed verification")
            }
        }
    }

    private func observeTransactionUpdates() -> Task<Void, Error> {
        Task.detached {
            for await verificationResult in Transaction.updates {
                do {
                    let transaction = try self.checkVerified(verificationResult)
                    await self.updatePurchasedProducts()
                    await transaction.finish()
                } catch {
                    print("Transaction failed verification")
                }
            }
        }
    }
}

enum StoreError: Error {
    case failedVerification
```

```
    case productNotFound
    case purchaseFailed
}
```

Create Receipt Validator

- Implement local receipt validation
- /FonicHiFi/Core/Subscription/ReceiptValidator.swift
- Create

```
import StoreKit

struct ReceiptValidator {
    func validateReceipt() async -> Bool {
        do {
            // Local validation using StoreKit 2
            let verificationResult = try await AppTransaction.shared

            switch verificationResult {
            case .verified(let appTransaction):
                // Receipt is valid
                return true
            case .unverified:
                // Receipt validation failed
                return false
            }
        } catch {
            print("Receipt validation error: \(error)")
            return false
        }
    }

    func validateSubscription(productID: String) async -> Bool {
        do {
            for await result in Transaction.currentEntitlements {
                do {
                    let transaction = try checkVerified(result)

                    if transaction.productID == productID && transaction.revocationDate
== nil {
                        // Check if subscription is still valid
                        if let expirationDate = transaction.expirationDate {
                            return Date() < expirationDate
                        }
                        return true
                    }
                } catch {
                    print("Transaction failed verification")
                }
            }
```

```
            return false
        } catch {
            print("Subscription validation error: \(error)")
            return false
        }
    }

    private func checkVerified<T>(_ result: VerificationResult<T>) throws -> T {
        switch result {
        case .unverified:
            throw StoreError.failedVerification
        case .verified(let safe):
            return safe
        }
    }
}
```

Implement Subscription Service

- Create service for managing subscriptions
- /FonicHiFi/Core/Subscription/SubscriptionService.swift
- Create

```
import StoreKit
import Combine

class SubscriptionService: ObservableObject {
    @Published private(set) var currentTier: SubscriptionTier = .free
    @Published private(set) var products: [SubscriptionProduct] = []
    @Published private(set) var isLoading = false

    private let storeService: StoreService
    private let receiptValidator: ReceiptValidator
    private var cancellables = Set<AnyCancellable>()

    init(storeService: StoreService, receiptValidator: ReceiptValidator) {
        self.storeService = storeService
        self.receiptValidator = receiptValidator

        // Load initial state
        Task {
            await loadProducts()
            await validateSubscriptionStatus()
        }
    }

    func loadProducts() async {
        await MainActor.run { isLoading = true }

        do {
```

```
            try await storeService.loadProducts()

            // Map StoreKit products to our model
            let storeKitProducts = await storeService.products

            await MainActor.run {
                self.products = SubscriptionConfiguration.defaultProducts.map { product
in
                    var updatedProduct = product
                    updatedProduct.storeKitProduct = storeKitProducts.first(where: { $0.id
== product.id })
                    return updatedProduct
                }
                isLoading = false
            }
        } catch {
            print("Failed to load products: \(error)")
            await MainActor.run { isLoading = false }
        }
    }

    func purchase(product: SubscriptionProduct) async -> Bool {
        guard let storeKitProduct = product.storeKitProduct else {
            return false
        }

        do {
            let transaction = try await storeService.purchase(storeKitProduct)
            if transaction != nil {
                await validateSubscriptionStatus()
                return true
            }
            return false
        } catch {
            print("Purchase failed: \(error)")
            return false
        }
    }

    func validateSubscriptionStatus() async {
        let premiumProductIDs = SubscriptionConfiguration.defaultProducts
            .filter { $0.tier == .premium }
            .map { $0.id }

        for productID in premiumProductIDs {
            let isSubscribed = await receiptValidator.validateSubscription(productID:
productID)
            if isSubscribed {
                await MainActor.run { currentTier = .premium }
                return
            }
        }
```

```
            await MainActor.run { currentTier = .free }
        }

        func canAccess(feature: FeatureFlag) -> Bool {
            return feature.requiredTier.rawValue <= currentTier.rawValue
        }

        func restorePurchases() async {
            do {
                try await AppStore.sync()
                await storeService.updatePurchasedProducts()
                await validateSubscriptionStatus()
            } catch {
                print("Failed to restore purchases: \(error)")
            }
        }
    }
```

Other Notes On Step 2: StoreKit 2 Integration

- Test with StoreKit testing configuration during development
- Ensure proper error handling for network issues or interrupted purchases
- This implementation uses local receipt validation; consider adding server-side validation for production

Step 3: Feature Gating Implementation

Detailed technical explanation of what we're accomplishing in this step

We'll implement the feature gating system throughout the app, ensuring premium features are only accessible to subscribers. This will involve updating the app state, service provider, and relevant view models to check subscription status before allowing access to premium features.

Task Breakdown

Update App State

- Add subscription state to app state
- /FonicHiFi/Presentation/UIState/AppState.swift
- Update

```
class AppState: ObservableObject {
    // Existing properties...

    @Published var subscriptionTier: SubscriptionTier = .free
```

```
    // Inject subscription service
    private let subscriptionService: SubscriptionService
    private var cancellables = Set<AnyCancellable>()

    init(subscriptionService: SubscriptionService, /* other dependencies */) {
        self.subscriptionService = subscriptionService
        // Other initialization...

        // Observe subscription changes
        subscriptionService.$currentTier
            .receive(on: RunLoop.main)
            .sink { [weak self] tier in
                self?.subscriptionTier = tier
            }
            .store(in: &cancellables)
    }

    func canAccess(feature: FeatureFlag) -> Bool {
        return subscriptionService.canAccess(feature: feature)
    }
}
```

Update Service Provider

- Add subscription services to dependency injection
- /FonicHiFi/App/ServiceProvider.swift
- Update

```
class ServiceProvider {
    // Singleton instance
    static let shared = ServiceProvider()

    // Store services
    lazy var storeService: StoreService = {
        return StoreService()
    }()

    lazy var receiptValidator: ReceiptValidator = {
        return ReceiptValidator()
    }()

    lazy var subscriptionService: SubscriptionService = {
        return SubscriptionService(
            storeService: storeService,
            receiptValidator: receiptValidator
        )
    }()

    // Existing services...
```

```
    // App state with subscription service
    lazy var appState: AppState = {
        return AppState(
            subscriptionService: subscriptionService,
            // Other dependencies...
        )
    }()
}
```

Create Feature Access Modifier

- Implement SwiftUI view modifier for feature gating
- /FonicHiFi/Presentation/Common/ViewModifiers/FeatureGatingModifier.swift
- Create

```
import SwiftUI

struct FeatureGatingModifier: ViewModifier {
    @EnvironmentObject private var appState: AppState

    let feature: FeatureFlag
    let alternativeView: AnyView?

    init(feature: FeatureFlag, alternativeView: AnyView? = nil) {
        self.feature = feature
        self.alternativeView = alternativeView
    }

    func body(content: Content) -> some View {
        Group {
            if appState.canAccess(feature: feature) {
                content
            } else {
                alternativeView ?? AnyView(
                    FeatureLockedView(feature: feature)
                )
            }
        }
    }
}

extension View {
    func gatedFeature(_ feature: FeatureFlag) -> some View {
        self.modifier(FeatureGatingModifier(feature: feature))
    }

    func gatedFeature(_ feature: FeatureFlag, alternativeView: some View) -> some
View {
        self.modifier(FeatureGatingModifier(
```

```
            feature: feature,
            alternativeView: AnyView(alternativeView)
        ))
    }
}
```

Implement Feature Locked View

- Create view for locked premium features
- /FonicHiFi/Presentation/Views/Common/FeatureLockedView.swift
- Create

```
import SwiftUI

struct FeatureLockedView: View {
    let feature: FeatureFlag
    @State private var showSubscription = false

    var body: some View {
        VStack(spacing: 16) {
            Image(systemName: "lock.fill")
                .font(.system(size: 40))
                .foregroundColor(.secondary)

            Text(feature.displayName)
                .font(.headline)

            Text("This feature requires a Premium subscription")
                .font(.subheadline)
                .foregroundColor(.secondary)
                .multilineTextAlignment(.center)
                .padding(.horizontal)

            Button("Upgrade to Premium") {
                showSubscription = true
            }
            .buttonStyle(.borderedProminent)
            .padding(.top, 8)
        }
        .padding()
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(Color(.systemBackground))
        .sheet(isPresented: $showSubscription) {
            SubscriptionView()
        }
    }
}
```

Gate Advanced DSP Features

- Update DSP chain to check subscription status
- /FonicHiFi/Core/Audio/AudioProcessing/DSPChain.swift
- Update

```
class DSPChain {
    private let subscriptionService: SubscriptionService

    init(subscriptionService: SubscriptionService) {
        self.subscriptionService = subscriptionService
    }

    func buildProcessingChain() -> [AudioProcessor] {
        var processors: [AudioProcessor] = []

        // Basic processors (available to all)
        processors.append(BasicEqualizerProcessor())
        processors.append(GainControlProcessor())

        // Advanced processors (premium only)
        if subscriptionService.canAccess(feature: .advancedDSP) {
            processors.append(ReplayGainProcessor())
            processors.append(CrossfadeProcessor())
            processors.append(SpatialAudioProcessor())
            // Other advanced processors...
        }

        return processors
    }
}
```

Other Notes On Step 3: Feature Gating Implementation

- Ensure feature gating is applied consistently across the app
- Consider caching feature access results for performance
- Test all feature gates with both subscription tiers

Step 4: Subscription UI Implementation

Detailed technical explanation of what we're accomplishing in this step

We'll implement the user interface for managing subscriptions, including a paywall, feature comparison, and upgrade prompts. This will provide a clear and compelling presentation of the subscription value proposition while maintaining the app's privacy-first approach.

Task Breakdown

Create Subscription View Model

- Implement view model for subscription management
- /FonicHiFi/Presentation/ViewModels/Subscription/SubscriptionViewModel.swift
- Create

```
import Combine
import StoreKit

class SubscriptionViewModel: ObservableObject {
    @Published var products: [SubscriptionProduct] = []
    @Published var currentTier: SubscriptionTier = .free
    @Published var isLoading = false
    @Published var purchaseInProgress = false
    @Published var errorMessage: String?

    private let subscriptionService: SubscriptionService
    private var cancellables = Set<AnyCancellable>()

    init(subscriptionService: SubscriptionService) {
        self.subscriptionService = subscriptionService

        subscriptionService.$products
            .receive(on: RunLoop.main)
            .assign(to: \.products, on: self)
            .store(in: &cancellables)

        subscriptionService.$currentTier
            .receive(on: RunLoop.main)
            .assign(to: \.currentTier, on: self)
            .store(in: &cancellables)

        subscriptionService.$isLoading
            .receive(on: RunLoop.main)
            .assign(to: \.isLoading, on: self)
            .store(in: &cancellables)

        // Load products
        Task {
            await subscriptionService.loadProducts()
        }
    }

    func purchase(product: SubscriptionProduct) {
        Task {
            purchaseInProgress = true
            let success = await subscriptionService.purchase(product: product)
```

```
            await MainActor.run {
                purchaseInProgress = false
                if !success {
                    errorMessage = "Purchase could not be completed. Please try again."
                }
            }
        }
    }

    func restorePurchases() {
        Task {
            isLoading = true
            await subscriptionService.restorePurchases()
            await MainActor.run { isLoading = false }
        }
    }

    var premiumFeatures: [FeatureFlag] {
        return FeatureFlag.allCases.filter { $0.requiredTier == .premium }
    }

    var freeFeatures: [FeatureFlag] {
        return FeatureFlag.allCases.filter { $0.requiredTier == .free }
    }
}
```

Implement Subscription View

- Create main subscription view with feature comparison
- /FonicHiFi/Presentation/Views/Subscription/SubscriptionView.swift
- Create

```
import SwiftUI

struct SubscriptionView: View {
    @StateObject private var viewModel = SubscriptionViewModel(
        subscriptionService: ServiceProvider.shared.subscriptionService
    )
    @Environment(\.dismiss) private var dismiss

    var body: some View {
        NavigationView {
            ScrollView {
                VStack(spacing: 24) {
                    headerView

                    featureComparisonView

                    subscriptionOptionsView
```

```
                footerView
            }
            .padding()
        }
        .navigationTitle("Fonic HiFi Premium")
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .navigationBarTrailing) {
                Button("Close") {
                    dismiss()
                }
            }
        }
        .overlay {
            if viewModel.isLoading || viewModel.purchaseInProgress {
                ProgressView()
                    .scaleEffect(1.5)
                    .frame(maxWidth: .infinity, maxHeight: .infinity)
                    .background(Color.black.opacity(0.2))
            }
        }
        .alert("Error", isPresented: Binding(
            get: { viewModel.errorMessage != nil },
            set: { if !$0 { viewModel.errorMessage = nil } }
        )) {
            Button("OK") { viewModel.errorMessage = nil }
        } message: {
            Text(viewModel.errorMessage ?? "")
        }
    }
}

private var headerView: some View {
    VStack(spacing: 16) {
        Image("premium_icon")
            .resizable()
            .scaledToFit()
            .frame(width: 80, height: 80)

        Text("Upgrade to Premium")
            .font(.title)
            .fontWeight(.bold)

        Text("Unlock advanced audio features while maintaining complete privacy")
            .font(.subheadline)
            .multilineTextAlignment(.center)
            .foregroundColor(.secondary)
            .padding(.horizontal)
    }
}

private var featureComparisonView: some View {
```

```
        VStack(spacing: 16) {
            Text("Premium Features")
                .font(.headline)
                .frame(maxWidth: .infinity, alignment: .leading)

            ForEach(viewModel.premiumFeatures, id: \.rawValue) { feature in
                HStack(spacing: 12) {
                    Image(systemName: "checkmark.circle.fill")
                        .foregroundColor(.green)

                    VStack(alignment: .leading, spacing: 4) {
                        Text(feature.displayName)
                            .font(.subheadline)
                            .fontWeight(.medium)

                        Text(feature.description)
                            .font(.caption)
                            .foregroundColor(.secondary)
                    }

                    Spacer()
                }
                .padding(.vertical, 4)
            }

            Divider()
                .padding(.vertical, 8)

            Text("Core Features (Included)")
                .font(.headline)
                .frame(maxWidth: .infinity, alignment: .leading)

            ForEach(viewModel.freeFeatures, id: \.rawValue) { feature in
                HStack(spacing: 12) {
                    Image(systemName: "checkmark.circle.fill")
                        .foregroundColor(.blue)

                    VStack(alignment: .leading, spacing: 4) {
                        Text(feature.displayName)
                            .font(.subheadline)
                            .fontWeight(.medium)

                        Text(feature.description)
                            .font(.caption)
                            .foregroundColor(.secondary)
                    }

                    Spacer()
                }
                .padding(.vertical, 4)
            }
        }
```

```
        .padding()
        .background(
            RoundedRectangle(cornerRadius: 12)
                .fill(Color(.secondarySystemBackground))
        )
    }

    private var subscriptionOptionsView: some View {
        VStack(spacing: 16) {
            Text("Choose Your Plan")
                .font(.headline)
                .frame(maxWidth: .infinity, alignment: .leading)

            ForEach(viewModel.products) { product in
                Button {
                    viewModel.purchase(product: product)
                } label: {
                    HStack {
                        VStack(alignment: .leading, spacing: 4) {
                            Text(product.displayName)
                                .font(.subheadline)
                                .fontWeight(.semibold)

                            Text(product.description)
                                .font(.caption)
                                .foregroundColor(.secondary)
                        }

                        Spacer()

                        if let storeKitProduct = product.storeKitProduct {
                            Text(storeKitProduct.displayPrice)
                                .font(.headline)
                                .fontWeight(.bold)
                        } else {
                            Text("$\(String(format: "%.2f", product.price))")
                                .font(.headline)
                                .fontWeight(.bold)
                        }
                    }
                    .padding()
                    .background(
                        RoundedRectangle(cornerRadius: 12)
                            .stroke(Color.accentColor, lineWidth: 2)
                    )
                }
                .buttonStyle(PlainButtonStyle())
            }
        }
    }

    private var footerView: some View {
```

```
        VStack(spacing: 16) {
            Button("Restore Purchases") {
                viewModel.restorePurchases()
            }
            .font(.subheadline)
            .foregroundColor(.accentColor)

            VStack(spacing: 8) {
                Text("Privacy Commitment")
                    .font(.subheadline)
                    .fontWeight(.medium)

                Text("Fonic HiFi never collects or shares your listening data. Your subscription is managed anonymously through Apple, with no account creation required.")
                    .font(.caption)
                    .foregroundColor(.secondary)
                    .multilineTextAlignment(.center)
            }
            .padding()
            .background(
                RoundedRectangle(cornerRadius: 12)
                    .fill(Color(.tertiarySystemBackground))
            )
        }
    }
}
```

Create Premium Feature Indicator

- Implement visual indicator for premium features
- /FonicHiFi/Presentation/Views/Common/PremiumFeatureIndicator.swift
- Create

```
import SwiftUI

struct PremiumFeatureIndicator: View {
    let feature: FeatureFlag

    var body: some View {
        HStack(spacing: 4) {
            Image(systemName: "star.fill")
                .font(.system(size: 10))

            Text("PREMIUM")
                .font(.system(size: 10, weight: .semibold))
        }
        .foregroundColor(.white)
        .padding(.horizontal, 6)
        .padding(.vertical, 2)
```

```
            .background(
                Capsule()
                    .fill(Color.yellow)
            )
        }
    }
```

Implement Feature Preview

- Create preview system for premium features
- /FonicHiFi/Presentation/Views/Common/FeaturePreviewView.swift
- Create

```
import SwiftUI

struct FeaturePreviewView<Content: View>: View {
    let feature: FeatureFlag
    let previewContent: Content
    @State private var showSubscription = false

    init(feature: FeatureFlag, @ViewBuilder previewContent: () -> Content) {
        self.feature = feature
        self.previewContent = previewContent()
    }

    var body: some View {
        VStack(spacing: 0) {
            previewContent
                .blur(radius: 3)
                .overlay(
                    Button {
                        showSubscription = true
                    } label: {
                        VStack(spacing: 12) {
                            PremiumFeatureIndicator(feature: feature)
                                .padding(.bottom, 4)

                            Text("Tap to unlock \(feature.displayName)")
                                .font(.headline)
                                .foregroundColor(.white)
                                .shadow(radius: 2)

                            Text("Premium Feature")
                                .font(.subheadline)
                                .foregroundColor(.white.opacity(0.8))
                                .shadow(radius: 2)
                        }
                        .padding()
                        .frame(maxWidth: .infinity, maxHeight: .infinity)
                        .background(Color.black.opacity(0.4))
```

```
            }
        )
    }
    .sheet(isPresented: $showSubscription) {
        SubscriptionView()
    }
  }
}
```

Update Settings View

- Add subscription management to settings
- /FonicHiFi/Presentation/Views/Settings/SettingsView.swift
- Update

```
struct SettingsView: View {
  @EnvironmentObject private var appState: AppState
  @State private var showSubscription = false

  var body: some View {
    Form {
      // Existing sections...

      Section(header: Text("Subscription")) {
        HStack {
          Text("Current Plan")
          Spacer()
          Text(appState.subscriptionTier.displayName)
            .foregroundColor(.secondary)
        }

        if appState.subscriptionTier == .free {
          Button("Upgrade to Premium") {
            showSubscription = true
          }
          .foregroundColor(.accentColor)
        } else {
          Button("Manage Subscription") {
            if let url = URL(string: "https://apps.apple.com/account/
subscriptions") {
              UIApplication.shared.open(url)
            }
          }
          .foregroundColor(.accentColor)
        }

        Button("Restore Purchases") {
          Task {
            await ServiceProvider.shared.subscriptionService.restorePurchases()
          }
```

```
            }
            .foregroundColor(.accentColor)
        }

        // Other sections...
      }
      .navigationTitle("Settings")
      .sheet(isPresented: $showSubscription) {
        SubscriptionView()
      }
    }
  }
```

Other Notes On Step 4: Subscription UI Implementation

- Ensure subscription UI follows Apple's Human Interface Guidelines
- Test with various screen sizes and accessibility settings
- Consider A/B testing different subscription presentations if possible

Step 5: Feature Gating Integration

Detailed technical explanation of what we're accomplishing in this step

We'll integrate the feature gating system with specific features throughout the app, ensuring premium features are properly restricted while providing clear upgrade paths. This will involve updating various view models and views to check subscription status and show appropriate UI.

Task Breakdown

Gate Cloud Integration Features

- Update cloud data sources to check subscription
- /FonicHiFi/Data/DataSources/Remote/CloudDataSource.swift
- Update

```
class CloudDataSource {
  private let subscriptionService: SubscriptionService

  init(subscriptionService: SubscriptionService) {
    self.subscriptionService = subscriptionService
  }

  func connect(to service: CloudService) async throws -> Bool {
    guard subscriptionService.canAccess(feature: .cloudIntegration) else {
      throw CloudError.premiumFeatureRequired
    }
```

```
        // Existing connection logic...
    }

    func listFiles(in path: String) async throws -> [CloudFile] {
        guard subscriptionService.canAccess(feature: .cloudIntegration) else {
            throw CloudError.premiumFeatureRequired
        }

        // Existing file listing logic...
    }

    // Other methods with similar checks...
}

enum CloudError: Error {
    case premiumFeatureRequired
    // Other errors...
}
```

Gate Advanced Metadata Features

- Update metadata editor to check subscription
- /FonicHiFi/Domain/UseCases/Metadata/BatchEditUseCase.swift
- Update

```
class BatchEditUseCase {
    private let metadataRepository: MetadataRepository
    private let subscriptionService: SubscriptionService

    init(metadataRepository: MetadataRepository, subscriptionService:
SubscriptionService) {
        self.metadataRepository = metadataRepository
        self.subscriptionService = subscriptionService
    }

    func execute(changes: MetadataChanges, tracks: [Track]) async throws ->
[Track] {
        guard subscriptionService.canAccess(feature: .advancedMetadata) else {
            throw MetadataError.premiumFeatureRequired
        }

        // Existing batch edit logic...
    }
}

enum MetadataError: Error {
    case premiumFeatureRequired
```

```
    // Other errors...
}
```

Gate High-Resolution Format Support

- Update format decoders to check subscription
- /FonicHiFi/Core/Audio/AudioEngine/EngineSelector.swift
- Update

```
class EngineSelector {
    private let subscriptionService: SubscriptionService

    init(subscriptionService: SubscriptionService) {
        self.subscriptionService = subscriptionService
    }

    func selectEngineFor(format: AudioFormat) -> AudioEngine {
        // Check if high-resolution format requires premium
        if format.isHighResolution && !
subscriptionService.canAccess(feature: .highResFormats) {
            // Fall back to standard quality engine
            return AVAudioEngineAdapter()
        }

        // Existing engine selection logic...
        switch format.fileType {
        case .dsd:
            // DSD requires premium
            if subscriptionService.canAccess(feature: .highResFormats) {
                return SFBAudioEngineAdapter()
            } else {
                // Fall back or throw error
                return AVAudioEngineAdapter()
            }
        case .flac, .alac:
            if format.bitDepth > 16 || format.sampleRate > 48000 {
                // High-res FLAC/ALAC requires premium
                if subscriptionService.canAccess(feature: .highResFormats) {
                    return SFBAudioEngineAdapter()
                } else {
                    // Fall back to standard quality
                    return AVAudioEngineAdapter()
                }
            } else {
                return AVAudioEngineAdapter()
            }
        default:
            return AVAudioEngineAdapter()
        }
```

```
    }
}
```

Gate Advanced Waveform Features

- Update waveform generator to check subscription
- /FonicHiFi/Core/Audio/Analysis/WaveformGenerator.swift
- Update

```
class WaveformGenerator {
    private let subscriptionService: SubscriptionService

    init(subscriptionService: SubscriptionService) {
        self.subscriptionService = subscriptionService
    }

    func generateWaveform(for track: Track, quality: WaveformQuality = .standard)
async throws -> Waveform {
        // Check if high-quality waveform requires premium
        let effectiveQuality: WaveformQuality

        if quality == .high &&
!subscriptionService.canAccess(feature: .advancedWaveforms) {
            // Fall back to standard quality
            effectiveQuality = .standard
        } else {
            effectiveQuality = quality
        }

        // Existing waveform generation logic with effectiveQuality...
    }
}

enum WaveformQuality {
    case low
    case standard
    case high
}
```

Update Player View Model

- Gate advanced player features
- /FonicHiFi/Presentation/ViewModels/Player/PlayerViewModel.swift
- Update

```
class PlayerViewModel: ObservableObject {
    // Existing properties...
```

```
    private let subscriptionService: SubscriptionService

    init(playbackRepository: PlaybackRepository, subscriptionService:
SubscriptionService) {
        self.playbackRepository = playbackRepository
        self.subscriptionService = subscriptionService
        // Other initialization...
    }

    // Check for advanced waveform access
    var canShowDetailedWaveform: Bool {
        return subscriptionService.canAccess(feature: .advancedWaveforms)
    }

    // Check for advanced DSP access
    var canAccessAdvancedDSP: Bool {
        return subscriptionService.canAccess(feature: .advancedDSP)
    }

    // Existing methods...
}
```

Other Notes On Step 5: Feature Gating Integration

- Ensure graceful degradation when premium features are not available
- Provide clear feedback when a user attempts to access a premium feature
- Test all feature gates with both subscription tiers

Step 6: Testing and Validation

Detailed technical explanation of what we're accomplishing in this step

We'll implement comprehensive testing for the subscription system, ensuring it works correctly across different scenarios. This will include unit tests, integration tests, and manual testing procedures to validate the subscription flow and feature gating.

Task Breakdown

Create Subscription Service Tests

- Implement unit tests for subscription service
- /FonicHiFi/Tests/UnitTests/Core/Subscription/SubscriptionServiceTests.swift
- Create

```
 import XCTest
@testable import FonicHiFi
import StoreKit
```

```
class SubscriptionServiceTests: XCTestCase {
    var mockStoreService: MockStoreService!
    var mockReceiptValidator: MockReceiptValidator!
    var subscriptionService: SubscriptionService!

    override func setUp() {
        super.setUp()
        mockStoreService = MockStoreService()
        mockReceiptValidator = MockReceiptValidator()
        subscriptionService = SubscriptionService(
            storeService: mockStoreService,
            receiptValidator: mockReceiptValidator
        )
    }

    override func tearDown() {
        mockStoreService = nil
        mockReceiptValidator = nil
        subscriptionService = nil
        super.tearDown()
    }

    func testInitialTierIsFree() {
        XCTAssertEqual(subscriptionService.currentTier, .free)
    }

    func testCanAccessFreeFeatures() {
        XCTAssertTrue(subscriptionService.canAccess(feature: .basicPlayback))
        XCTAssertTrue(subscriptionService.canAccess(feature: .standardFormats))
        XCTAssertTrue(subscriptionService.canAccess(feature: .basicEqualizer))
        XCTAssertTrue(subscriptionService.canAccess(feature: .localLibrary))
    }

    func testCannotAccessPremiumFeaturesWithFreeTier() {
        XCTAssertFalse(subscriptionService.canAccess(feature: .advancedDSP))
        XCTAssertFalse(subscriptionService.canAccess(feature: .cloudIntegration))
        XCTAssertFalse(subscriptionService.canAccess(feature: .advancedMetadata))
        XCTAssertFalse(subscriptionService.canAccess(feature: .highResFormats))
        XCTAssertFalse(subscriptionService.canAccess(feature: .advancedWaveforms))
    }

    func testCanAccessAllFeaturesWithPremiumTier() async {
        // Simulate premium subscription
        mockReceiptValidator.isSubscriptionValid = true
        await subscriptionService.validateSubscriptionStatus()

        XCTAssertEqual(subscriptionService.currentTier, .premium)

        // Should have access to all features
        for feature in FeatureFlag.allCases {
            XCTAssertTrue(subscriptionService.canAccess(feature: feature))
```

```
    }
}

func testPurchaseSuccess() async {
    // Setup mock product
    let product = SubscriptionProduct(
        id: "test.premium.monthly",
        displayName: "Test Premium",
        description: "Test",
        price: 4.99,
        tier: .premium,
        duration: .monthly
    )

    // Configure mock for successful purchase
    mockStoreService.purchaseSuccess = true

    // Attempt purchase
    let result = await subscriptionService.purchase(product: product)

    XCTAssertTrue(result)
    XCTAssertEqual(mockStoreService.purchasedProductID, product.id)
}

func testPurchaseFailure() async {
    // Setup mock product
    let product = SubscriptionProduct(
        id: "test.premium.monthly",
        displayName: "Test Premium",
        description: "Test",
        price: 4.99,
        tier: .premium,
        duration: .monthly
    )

    // Configure mock for failed purchase
    mockStoreService.purchaseSuccess = false

    // Attempt purchase
    let result = await subscriptionService.purchase(product: product)

    XCTAssertFalse(result)
}

func testRestorePurchases() async {
    // Configure mock for successful restore
    mockReceiptValidator.isSubscriptionValid = true

    // Restore purchases
    await subscriptionService.restorePurchases()

    // Verify subscription status was validated
```

```
        XCTAssertTrue(mockReceiptValidator.validateSubscriptionCalled)
        XCTAssertEqual(subscriptionService.currentTier, .premium)
    }
}

// Mock implementations for testing
class MockStoreService: StoreService {
    var purchaseSuccess = false
    var purchasedProductID: String?

    override func purchase(_ product: Product) async throws -> Transaction? {
        if purchaseSuccess {
            purchasedProductID = product.id
            return nil // Simplified for testing
        } else {
            throw StoreError.purchaseFailed
        }
    }
}

class MockReceiptValidator: ReceiptValidator {
    var isSubscriptionValid = false
    var validateSubscriptionCalled = false

    override func validateSubscription(productID: String) async -> Bool {
        validateSubscriptionCalled = true
        return isSubscriptionValid
    }
}
```

Create Feature Gating Tests

- Implement tests for feature gating
- /FonicHiFi/Tests/UnitTests/Presentation/FeatureGatingTests.swift
- Create

```
 import XCTest
import SwiftUI
@testable import FonicHiFi

class FeatureGatingTests: XCTestCase {
    var mockSubscriptionService: MockSubscriptionService!
    var appState: AppState!

    override func setUp() {
        super.setUp()
        mockSubscriptionService = MockSubscriptionService()
        appState = AppState(subscriptionService: mockSubscriptionService)
    }
```

```
    override func tearDown() {
        mockSubscriptionService = nil
        appState = nil
        super.tearDown()
    }

    func testAppStateReflectsSubscriptionTier() {
        // Initial state should be free
        XCTAssertEqual(appState.subscriptionTier, .free)

        // Update subscription tier
        mockSubscriptionService.currentTier = .premium

        // Wait for publisher to update
        let expectation = XCTestExpectation(description: "Subscription tier updated")
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.1) {
            XCTAssertEqual(self.appState.subscriptionTier, .premium)
            expectation.fulfill()
        }

        wait(for: [expectation], timeout: 1.0)
    }

    func testFeatureAccessChecksSubscriptionService() {
        // Test free feature
        mockSubscriptionService.accessibleFeatures = [.basicPlayback]
        XCTAssertTrue(appState.canAccess(feature: .basicPlayback))
        XCTAssertFalse(appState.canAccess(feature: .advancedDSP))

        // Test premium feature
        mockSubscriptionService.accessibleFeatures = [.basicPlayback, .advancedDSP]
        XCTAssertTrue(appState.canAccess(feature: .basicPlayback))
        XCTAssertTrue(appState.canAccess(feature: .advancedDSP))
    }

    func testFeatureGatingModifier() {
        // Create a test view with the modifier
        let testView = Text("Test Content")
            .modifier(FeatureGatingModifier(feature: .advancedDSP))

        // With free tier, should show locked view
        mockSubscriptionService.accessibleFeatures = []
        // This would require ViewInspector or similar to fully test

        // With premium tier, should show content
        mockSubscriptionService.accessibleFeatures = [.advancedDSP]
        // This would require ViewInspector or similar to fully test
    }
}

// Mock implementation for testing
class MockSubscriptionService: SubscriptionService {
```

```
    var accessibleFeatures: [FeatureFlag] = []

    override func canAccess(feature: FeatureFlag) -> Bool {
        return accessibleFeatures.contains(feature)
    }
}
```

Create StoreKit Test Configuration

- Set up StoreKit testing configuration
- /FonicHiFi/Resources/StoreKitTestConfiguration.storekit
- Create

```
{
  "identifier" : "test_configuration",
  "products" : [
    {
      "displayPrice" : "4.99",
      "familyShareable" : false,
      "internalID" : "com.fonichifi.subscription.premium.monthly",
      "localizations" : [
        {
          "description" : "Full access to all premium features",
          "displayName" : "Premium Monthly",
          "locale" : "en_US"
        }
      ],
      "productID" : "com.fonichifi.subscription.premium.monthly",
      "referenceName" : "Premium Monthly",
      "type" : "Consumable"
    },
    {
      "displayPrice" : "49.99",
      "familyShareable" : false,
      "internalID" : "com.fonichifi.subscription.premium.yearly",
      "localizations" : [
        {
          "description" : "Full access to all premium features",
          "displayName" : "Premium Yearly",
          "locale" : "en_US"
        }
      ],
      "productID" : "com.fonichifi.subscription.premium.yearly",
      "referenceName" : "Premium Yearly",
      "type" : "Consumable"
    }
  ],
  "settings" : {

  },
```

```
"subscriptionGroups" : [
 {
  "id" : "premium_subscriptions",
  "localizations" : [
   {
    "description" : "Premium features for Fonic HiFi",
    "displayName" : "Fonic HiFi Premium",
    "locale" : "en_US"
   }
  ],
  "name" : "Premium Subscriptions",
  "subscriptions" : [
   {
    "adHocOffers" : [
     {
      "displayPrice" : "0",
      "internalID" : "trial_offer",
      "numberOfPeriods" : 1,
      "paymentMode" : "free",
      "subscriptionPeriod" : "P1W"
     }
    ],
    "codeOffers" : [

    ],
    "displayPrice" : "4.99",
    "familyShareable" : false,
    "groupNumber" : 1,
    "internalID" : "monthly_subscription",
    "introductoryOffer" : {
     "displayPrice" : "0",
     "internalID" : "intro_offer",
     "numberOfPeriods" : 1,
     "paymentMode" : "free",
     "subscriptionPeriod" : "P1W"
    },
    "localizations" : [
     {
      "description" : "Full access to all premium features",
      "displayName" : "Premium Monthly",
      "locale" : "en_US"
     }
    ],
    "productID" : "com.fonichifi.subscription.premium.monthly",
    "recurringSubscriptionPeriod" : "P1M",
    "referenceName" : "Premium Monthly",
    "subscriptionGroupID" : "premium_subscriptions",
    "type" : "RecurringSubscription"
   },
   {
    "adHocOffers" : [
```

```
        ],
        "codeOffers" : [

        ],
        "displayPrice" : "49.99",
        "familyShareable" : false,
        "groupNumber" : 1,
        "internalID" : "yearly_subscription",
        "introductoryOffer" : {
          "displayPrice" : "0",
          "internalID" : "intro_offer_yearly",
          "numberOfPeriods" : 1,
          "paymentMode" : "free",
          "subscriptionPeriod" : "P1W"
        },
        "localizations" : [
          {
            "description" : "Full access to all premium features",
            "displayName" : "Premium Yearly",
            "locale" : "en_US"
          }
        ],
        "productID" : "com.fonichifi.subscription.premium.yearly",
        "recurringSubscriptionPeriod" : "P1Y",
        "referenceName" : "Premium Yearly",
        "subscriptionGroupID" : "premium_subscriptions",
        "type" : "RecurringSubscription"
      }
    ]
  }
 ],
 "version" : {
   "major" : 2,
   "minor" : 0
 }
}
```


Create Manual Test Plan

- Document manual testing procedures
- /Documentation/SubscriptionTestPlan.md
- Create

```
# Subscription Testing Plan

This document outlines the manual testing procedures for the Fonic HiFi subscription system.

## Test Environment Setup
```

1. Use StoreKit testing configuration in Xcode
2. Enable StoreKit testing in the scheme
3. Use the StoreKit transaction manager in Xcode to simulate purchases

Test Cases

Basic Functionality

1. **Verify Free Tier Access**
   - Launch app with no subscription
   - Verify access to all free features
   - Verify premium features are locked

2. **Subscription Purchase Flow**
   - Navigate to subscription view
   - Verify all subscription options are displayed
   - Purchase a subscription using StoreKit testing
   - Verify subscription status updates
   - Verify access to premium features

3. **Restore Purchases**
   - Clear app state
   - Use "Restore Purchases" button
   - Verify subscription is restored
   - Verify premium features are accessible

Feature Gating

1. **Cloud Integration**
   - Attempt to connect to cloud service with free tier
   - Verify premium upgrade prompt is shown
   - Subscribe to premium
   - Verify cloud integration works

2. **Advanced Metadata Editing**
   - Attempt batch editing with free tier
   - Verify premium upgrade prompt is shown
   - Subscribe to premium
   - Verify batch editing works

3. **High-Resolution Formats**
   - Attempt to play DSD file with free tier
   - Verify fallback to standard quality or upgrade prompt
   - Subscribe to premium
   - Verify high-resolution playback works

4. **Advanced Waveforms**
   - View waveform with free tier
   - Verify standard quality waveform is shown
   - Subscribe to premium
   - Verify high-quality waveform is shown

5. **Advanced DSP**
   - Access DSP settings with free tier
   - Verify basic features are available
   - Verify advanced features show upgrade prompt
   - Subscribe to premium
   - Verify all DSP features are available

Edge Cases

1. **Subscription Expiration**
   - Purchase subscription
   - Use StoreKit testing to expire subscription
   - Verify app reverts to free tier
   - Verify premium features are locked

2. **Network Issues**
   - Enable airplane mode
   - Attempt to purchase subscription
   - Verify appropriate error handling
   - Restore network connection
   - Complete purchase
   - Verify subscription activates

3. **App Reinstallation**
   - Purchase subscription
   - Delete and reinstall app
   - Restore purchases
   - Verify subscription is restored

Regression Testing

After implementing the subscription system, verify that all core app functionality continues to work correctly:

1. Playback of all supported formats
2. Library browsing and organization
3. Playlist creation and management
4. Settings and preferences
5. UI responsiveness and animations

Accessibility Testing

Verify that the subscription UI is fully accessible:

1. Test with VoiceOver
2. Test with Dynamic Type (all sizes)
3. Test with Reduce Motion enabled
4. Test with High Contrast mode

Update Info.plist for StoreKit

- Configure app for in-app purchases
- /FonicHiFi/Resources/Info.plist
- Update

```
 <key>StoreKit</key>
<dict>
  <key>Configuration</key>
  <string>Default</string>
</dict>
```

Other Notes On Step 6: Testing and Validation

- Test with various subscription scenarios (purchase, restore, expiration)
- Verify proper error handling for network issues and interrupted purchases
- Ensure subscription UI is fully accessible

Implementation Notes

This SaaS implementation plan is designed to add a privacy-first subscription model to the Fonic HiFi app. The approach uses StoreKit 2 for anonymous subscription management, with no user accounts or data collection required. The implementation includes:

1. **Subscription Model Foundation**: Defining subscription tiers, feature flags, and product models.
2. **StoreKit 2 Integration**: Implementing the infrastructure for in-app purchases and subscriptions.
3. **Feature Gating Implementation**: Creating a system to restrict premium features to subscribers.
4. **Subscription UI Implementation**: Building a compelling and clear subscription interface.
5. **Feature Gating Integration**: Applying feature gating to specific features throughout the app.
6. **Testing and Validation**: Ensuring the subscription system works correctly across different scenarios.

The implementation maintains the app's privacy-first philosophy by: - Using device-based entitlements linked to Apple ID (no separate accounts) - Performing local receipt validation with optional server-side verification - Collecting no user data or analytics - Providing clear privacy messaging in the subscription UI

This approach creates a sustainable revenue model while respecting user privacy and providing clear value through premium features.